BTS CIEL-IR · SUITE DU COURS PHP

PHPSESSID

Cookies & Sessions

Comment un site « se souvient » d’un visiteur — et comment le faire sans ouvrir de faille.

Cookies | Sessions | Authentification | « Se souvenir de moi » | Expiration

00 · RAPPEL

HTTP n’a pas de mémoire



Observer : F12 → onglet Réseau (en-têtes Set-Cookie et Cookie), puis Application → Cookies ; ou curl -i http://localhost/page.php

Créer, lire et supprimer un cookie

```php
// Créer — AVANT tout affichage
setcookie("theme", "sombre", [
  "expires" => time() + 3600 * 24 * 30,
  "path" => "/",
  "httponly" => true,
  "samesite" => "Lax",
]);

// Lire — à la requête SUIVANTE
$theme = $_COOKIE["theme"] ?? "clair";
echo htmlspecialchars($theme, ENT_QUOTES, "UTF-8");

// Supprimer — date passée, même path
setcookie("theme", "", [
  "expires" => time() - 3600, "path" => "/"]);
```

Avant tout affichage

Le cookie part dans les en-têtes : un echo, un espace ou un include qui affiche avant, et c’est « headers already sent ».

Lisible au rechargement

Un cookie créé n’apparaît dans $_COOKIE qu’à la requête suivante.

Une donnée du client

L’utilisateur peut le lire et le modifier : valider, échapper à l’affichage, jamais de mot de passe ni de prix.

Les options d’un cookie

Option	Rôle	Valeur conseillée
expires	Date d’expiration (horodatage Unix) ; absente : supprimé à la fermeture du navigateur	time() + durée
path	Partie du site pour laquelle le cookie est envoyé	"/" (tout le site)
domain	Domaine concerné	Domaine courant
secure	Envoi en HTTPS uniquement	true en production
httponly	Invisible pour JavaScript : limite le vol par une faille XSS	true
samesite	Limite l’envoi depuis d’autres sites (protection partielle contre le CSRF)	"Lax" ou "Strict"

XAMPP active output_buffering : « headers already sent » peut rester caché en local, puis surgir sur un autre serveur.

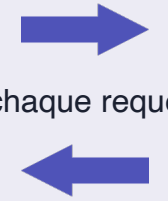
Limites : environ 4 Ko par cookie, renvoyé à chaque requête. Jamais de donnée sensible.

Des données gardées côté serveur

Navigateur

garde seulement un identifiant aléatoire

`PHPSESSID=7f3a9c…`



Serveur

garde les données de la session

`$_SESSION["pseudo"] = "Lucas"`

```
session_start([
  "cookie_httponly" => true,
  "cookie_samesite" => "Lax",
  "use_strict_mode" => true,
]);
$_SESSION["pseudo"] = "Lucas"; // stocker
unset($_SESSION["pseudo"]); // retirer
```

session_start() en premier

Avant tout affichage, idéalement en première ligne ou dans un auth.php inclus en premier.

use_strict_mode

Refuse un identifiant de session que le serveur ne connaît pas.

Déconnexion et durée de vie

Déconnexion complète : trois actions

```php
<?php
session_start();
$_SESSION = []; // 1. vider les données
setcookie(session_name(), "", [
  "expires" => time() - 3600, "path" => "/",
]); // 2. supprimer PHPSESSID
session_destroy(); // 3. côté serveur
header("Location: index.php");
exit;
```

> session_destroy() seule ne suffit pas : le cookie PHPSESSID resterait dans le navigateur.

Combien de temps dure une session ?

24 min

d'inactivité : le serveur supprime les données (gc_maxlifetime = 1440 s, nettoyage différé).

Fermeture

du navigateur : le cookie PHPSESSID disparaît (sauf restauration au redémarrage).

Pour une durée précise, gérer soi-même l'expiration par inactivité (exercice 5).

03 · CHOISIR

Cookie ou session ?

Critère	Cookie	Session
Stockage	Navigateur du client	Serveur (le client n'a que l'identifiant)
Durée	Fixée par expires, sinon fermeture du navigateur	Inactivité (gc_maxlifetime) et cookie PHPSESSID
Modifiable ?	Oui, par l'utilisateur : aucune confiance	Non ; l'identifiant reste à protéger
Taille	≈ 4 Ko	Peu limitée
Démarrage	Pas de session_start()	session_start() obligatoire

Cookie pour…

un thème, une langue, un pseudo : des préférences non sensibles.

Session pour…

l'authentification, le panier : tout ce dont le serveur doit être sûr.

Se connecter avec une session

```php
// login.php
session_start();
$login = trim($_POST["login"] ?? "");
$pass = $_POST["password"] ?? "";

if ($login === $compteTest["login"]
    && password_verify($pass, $compteTest["hash"])) {
  session_regenerate_id(true);
  $_SESSION["user"] = $login;
  header("Location: private.php");
  exit;
}
$erreur = "Identifiants incorrects.";
```

1. **password_verify()**
   Compare la saisie au hash stocké.
2. **session_regenerate_id(true)**
   Nouvel identifiant : bloque la fixation de session.
3. **Message générique**
   Ne pas révéler quel champ est faux.

```php
// private.php
if (!isset($_SESSION["user"])) {
  http_response_code(403);
  exit("Accès interdit.");
}
```

05 · « SE SOUVENIR DE MOI »

Un cookie ne prouve rien

Dangereux

```
setcookie("remember", $pseudo, time() + 604800);
// puis : cookie présent → « connecté »
```

remember=admin

Il suffit de créer ce cookie à la main pour être connecté en administrateur, **sans mot de passe** (extension E1 du TP3).

Correct : jeton aléatoire + empreinte

1. À la connexion : $token = bin2hex(random_bytes(32))
2. Le serveur garde hash("sha256", $token) ; le cookie contient pseudo:token (httponly, samesite).
3. Visite suivante sans session : comparer avec hash_equals(), puis session_regenerate_id(true).
4. Déconnexion : supprimer le jeton côté serveur **et** le cookie.

Protéger la session

Risque	Parade
Fixation de session : l'attaquant impose un identifiant connu	session_regenerate_id(true) dès la connexion ; use_strict_mode
Vol de l'identifiant (XSS, écoute du réseau)	Cookie httponly, secure + HTTPS ; échapper tous les affichages
Session ouverte indéfiniment	Expiration par inactivité (exercice 5)
Données sensibles	Rester sur le serveur ; ne stocker que le nécessaire

À vous : exercices 1 à 5, puis TP3 Authentification et ses extensions E1 à E6.